

JavaScript Outline

Mastering Variables And Data Types

JS Basics

- **Console** in javascript
- **Comments** in javascript
- Statements in javascript
- **Statement terminator** (;)

Variables In JavaScript

- How to **declare** a variable
- How to **initialize** a variable
- How to **use** a variable
- Constant variable
- Keyword **let** vs **var** vs **const**
- **Scope** of variables (local(function or block) scope **VS** global scope)
- **Rules** for variable names

Naming Conventions

- camelCase
- snake_case
- kabab-case
- PascalCase

Primitive Datatypes (NNSSBBU)

- Number
- Null

- String
- Symbol
- BigInt
- Boolean
- Undefined

JavaScript Is a Dynamically Typed Language

- **typeof()** Operator
- Variables dynamically change their data type according to the data assigned to them

JavaScript Is a Forgiving Language

- JavaScript is forgiving because it allows you to write code that still works even if you make small errors or overlook some rules.

Type Coercion

- Implicit type coercion
- Explicit type coercion
- To **Number**: Number(), parseInt(), parseFloat()
- To **String**: String(), .toString()
- To **Boolean**: Boolean()

Mastering JavaScript Operators

Basics Of Operators

- Operands vs operators
- What are JS **expressions**?
- **Precedence** and **associativity**

Types Of Operators

- **Assignment** operators (=, +=, -=, *=, /=, %=, **=)

- **Arithmetic** operators (+, -, *, /, %, **)
- **Comparison** operators (<, >, <=, >=, ==, !=, ===, !==)
- **Logical** operators (&&, ||, !)
- **Increment** or **Decrement** (++ , --)
- **Ternary** Operators (condition ? true : false)
- **Bitwise** operators (&, |, ~, <<, >>, ^)

Truthy And Falsy Values

- Falsy values (null, undefined, "", 0, NaN, false)
- Truthy values (anything that is not falsy)

Short Circuiting

- Short circuiting with logical AND(&&) operator
- Short circuiting with logical OR(||) operator
- Short circuiting with the nullish coalescing (??) operator

Mastering JavaScript Conditional Statements

Basics Of Conditional Statements

- What Are **Conditional** Statements
- **Use Cases** Of Conditionals In JavaScript
- Use Case of **Template Literals**(`\${}`)

Types Of Conditions

- **If** statement
- **If-else** statement
- **If-else-If** statement (also called **else-if** ladder)
- **Switch** statement
- **Ternary** operator
- Nesting with **if-else** and **ternary**

Mastering JavaScript Loops

Basics Of Loops

- What are loops in JS?
- Use cases of loops in JS?

Types Of Loops

- **For** loop
- **While** loop
- **Do-while** loop
- **For-of** loop (array, string, map, sets, [Don't care about index])
- **For-in** loop (iterating over properties of an object **or** the index of arrays)

Mastering JavaScript Functions

Basics Of Function

- What are the functions in JS?
- Function **definition**
- Function **call**/invoke
- Function **parameter** in JS
- Function **arguments** in JS
- **Default parameters** in functions
- Function **return** value
- What is meant by the **reference** to a function
- **Hoisting** in functions

4 Ways To Create Functions

- Without parameter list & without return value
- Without parameter list & with return value
- With parameter list & without return value

- With parameter list & with return value

Types Of Functions

- **Normal** function
- Function **expression**
- Fat **arrow** function
- **Anonymous** function
- **IIFE** (Immediately Invoked Function Expression)

Mastering JavaScript Strings

Basics Of Strings

- String in JS
- You can use **single** quotes(' '), **double** quotes(" ")
- String **literal method**: `let str = "Hussain";`
- String **constructor method**: `let str = new String("Hussain");`
- **Template literals**(` `)
- **String Interpolation** (` \${}`)
- String has **indexing** (position)
- Strings are **immutable**(can't be changed) in JS

Indexing In String

Positive Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Array	G	E	E	K	S	F	O	R	G	E	E	K	S
Negative Index	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

String Properties

- `str.length`

String Methods

- `str.toUpperCase()`
- `str.toLowerCase()`
- `str.trim()`
- `str.concat(str2)`
- `str.includes()`
- `str.indexOf()`
- `str.lastIndexOf()`
- `str.charAt()`
- `str.replace(old, new)`
- `str.replaceAll(old, new)`
- `str.slice(start, end)`
- `str.split()`
- `str.match(regExp)`

Mastering JavaScript Arrays

Basics Of Arrays

- Array **literal method**: `let arr = [val1, val2, ..., valN]`
- Array **constructor method**: `let arr = new Array(val1, val2, ..., valN)`
- **Nesting** with arrays. (2-D Array, multi-dimensional Array)
- An array has **indexing** (Positioning)
- Arrays are **mutable** (they can easily be changed)
- **typeof** array is not "array", it's an "**object**".

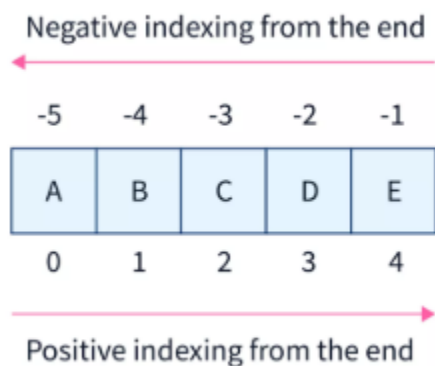
Difference Between Array Literal And Array Constructor

```
let arr = [20]
let carr = new Array(20).fill(0)
console.log(arr)
console.log(carr)
```

The array constructor is primarily used when we want to create an empty array with a **specified length**.

Indexing In Arrays

- let arr = ["A", "B", "C", "D", "E"];



Array Methods

Method	Changes the Original Array?	What It Returns	What It Does
push()	Yes	New length of the array	Add item at the end
pop()	Yes	The removed last item	Delete item from end
unshift()	Yes	New length of the array	Add item to start
shift()	Yes	The removed first item	Delete item from start
splice()	Yes	An array of removed items	Add remove or doing both at the same time
slice()	No	A new array (sliced portion)	Returns a piece of array
concat()	No	A new array (combined arrays)	Joins multiple arrays (we can also use spread operator instead)
join()	No	A string made by joining elements	join the elements of array on the basis of some string or character
toString()	No	A string of array elements	Converts array to string
includes()	No	true or false	Check whether given item is present in array or not
indexOf()	No	Index of the item or -1 if not found	returns the index of given item if it is present in array, otherwise returns -1
reverse()	Yes	The reversed array	Reverse the order of items
sort()	Yes	The sorted array	Sort the array
find()	No	The first matching item (or undefined)	Returns the first element in the array that satisfies a condition
flat()	No	A new array with nested arrays flattened	returns a new array with nested arrays flattened

Sorting An Array

- arr.sort(compareFunction)
- It works on **Tim Sort** under the hood (a combination of **merge sort** and **insertion sort**)

- By default, `sort()` converts elements to strings and sorts them alphabetically (**Unicode order**) or **Lexicographical order**.

Return Value of the function	Meaning	Effect on Order
▼ Negative (< 0)	a should come before b	✅ Keep order as-is
▬ Zero (=== 0)	a and b are equal	👉 Keep their order (stable)
▲ Positive (> 0)	a should come after b	🔄 Swap a and b

Following is an example of the compare function(for Ascending Order) 😊

```
let arr = [10, 5, 1, 20];
console.log(`Unsorted array: ${arr}`);

let sortedArr = arr.sort((a, b) => {
  console.log(`a = ${a} and b = ${b}`);
  return a - b;
});

console.log(`Sorted array: ${sortedArr}`);
```

Array Iteration Methods / Functional Array Methods

Method	Returns	Modifies Original Array?
<code>map()</code>	New array	No
<code>filter()</code>	New array	No
<code>reduce()</code>	Single value	No
<code>forEach()</code>	undefined	No (unless you do it manually)

Mastering JavaScript Objects

Basics Of Objects

- An object is a collection of **key-value** pairs.
- Object **Literal Method**: `let obj = { name: "Hussain", age: 20 };`
- Object **constructor Method**: `let obj = new Object({});`
- Objects are **mutable** (they can easily be changed)

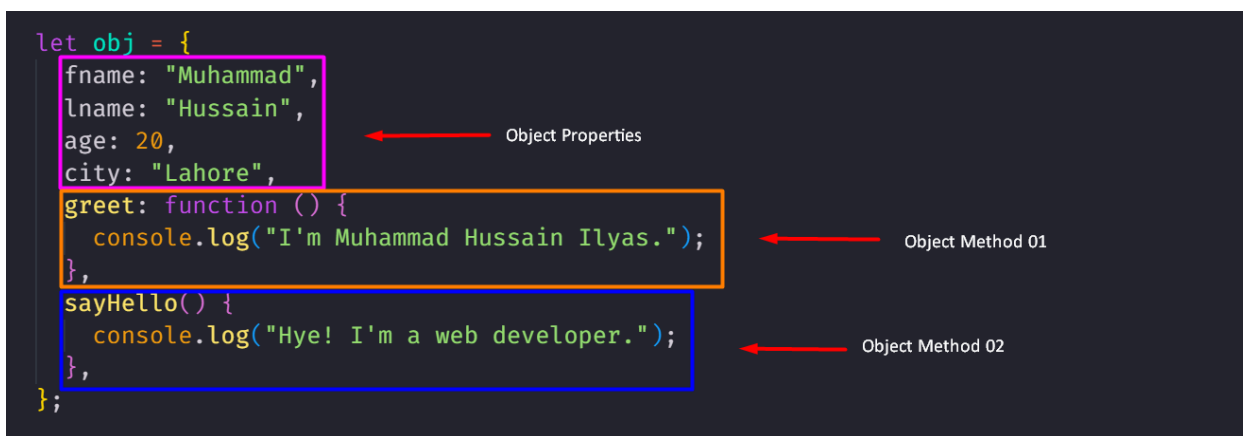
- **this** keyword in an object.

Accessing Object Properties

You can access properties in two ways:

- **Dot Notation:** `obj.prop`
- **Bracket Notation:** `obj["prop"]`

Object Properties vs Object Methods



```
let obj = {  
  fname: "Muhammad",  
  lname: "Hussain",  
  age: 20,  
  city: "Lahore",  
  greet: function () {  
    console.log("I'm Muhammad Hussain Ilyas.");  
  },  
  sayHello() {  
    console.log("Hye! I'm a web developer.");  
  },  
};
```

The diagram illustrates the structure of a JavaScript object. The object `obj` is defined with several properties and methods. A pink box highlights the properties: `fname: "Muhammad"`, `lname: "Hussain"`, `age: 20`, and `city: "Lahore"`. A red arrow points from the text "Object Properties" to this box. An orange box highlights the `greet` method, which is a function that logs a message. A red arrow points from the text "Object Method 01" to this box. A blue box highlights the `sayHello` method, which is also a function that logs a message. A red arrow points from the text "Object Method 02" to this box.

Adding/Updating/Deleting Object Properties

```
let obj = {  
  name: "Muhammad Hussain",  
  age: 19,  
};  
  
// Adding New Property  
obj.grade = "A";  
  
// Updating Existing Property  
obj.age = 20;  
  
// Deleting Property  
delete obj.name;
```

Object Methods

Method	Changes Original Object?	What It Returns	What It Does
Object.keys(obj)	✗ No	Array of keys	Returns an array of all enumerable property names (keys)
Object.values(obj)	✗ No	Array of values	Returns an array of all enumerable property values
Object.entries(obj)	✗ No	Array of [key, value]	Returns an array of [key, value] pairs
Object.assign(target, source)	✓ Yes (if target is modified)	Modified target object	Copies properties from source to target and returns the updated object
Object.freeze(obj)	✓ Yes (locks the object)	The frozen object	Prevents any changes (no adding, removing, or modifying properties)
Object.seal(obj)	✓ Yes (locks structure)	The sealed object	Prevents adding/removing properties, but allows modifying existing ones
ObjName.hasOwnProperty(key)	✗ No	Boolean (true / false)	Checks if the object has the specified property directly

Singleton Object vs Class-Based Object

Singleton Object

A singleton object is a single instance of an object that is globally accessible. It is created once and used throughout the application. Singleton patterns are useful when only one instance of an object is needed.

For Example

```
const singletonObj = {
  name: "Muhammad Hussain",
  grade: "A",
  age: 19,
  log: function (message) {
    console.log(`${message}! 😊 ${this.name}`);
  },
};

singletonObj.log("Hello");
```

Class-Based Object

In ES6, JavaScript introduced classes as syntactic sugar over constructor functions. Classes let you define a blueprint for creating objects with the same properties and methods, and you can create multiple instances from a class.

For Example

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet = function () {
    console.log(`Hye! I'm ${this.name} and I'm ${this.age} years old.`);
  };
}

let personOne = new Person("Muhammad Hussain", 20);
let personTwo = new Person("Suleman Amjad", 21);

console.log(personOne);
console.log(personTwo);
```

Destructuring

- **Object** Destructuring
- **Array** Destructuring
- **Function** Destructuring

Array Of Objects

An array whose values are objects is called an **Array of Objects**.

```
const products = [
  { id: 1, name: "iPhone 14" },
  { id: 2, name: "Samsung Galaxy" },
  { id: 3, name: "Google Pixel" },
];
```

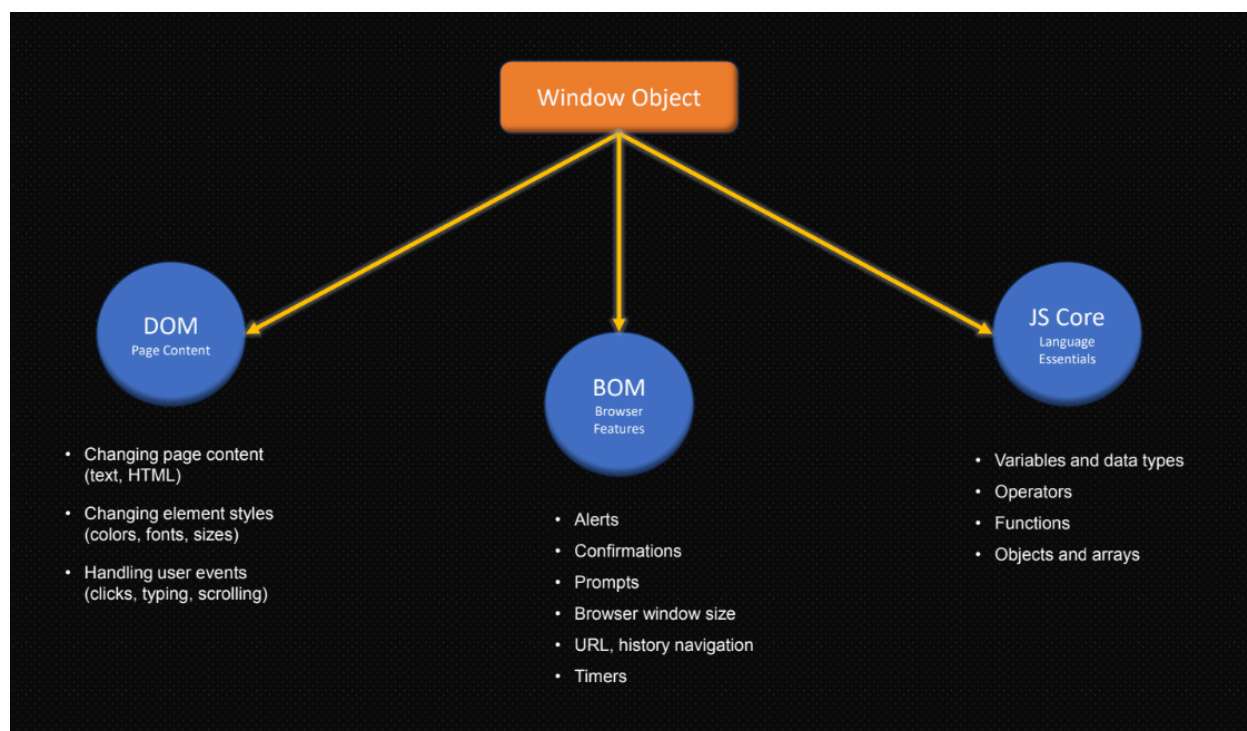
Mastering DOM Manipulation

Five(5) Phases Of DOM Manipulation

- What is **DOM**

- **Selecting HTML** elements
- How to change/**manipulate HTML**
- How to change/**manipulate CSS**
- **Event** Listeners (Event handling)

Window Object



The window object in JavaScript is the **global object**. It acts as the main interface to the browser's environment and provides properties, methods, and events that you can use to interact with the webpage and the browser itself. In the global scope, you can often **omit window**, because its properties and methods are **globally accessible**.

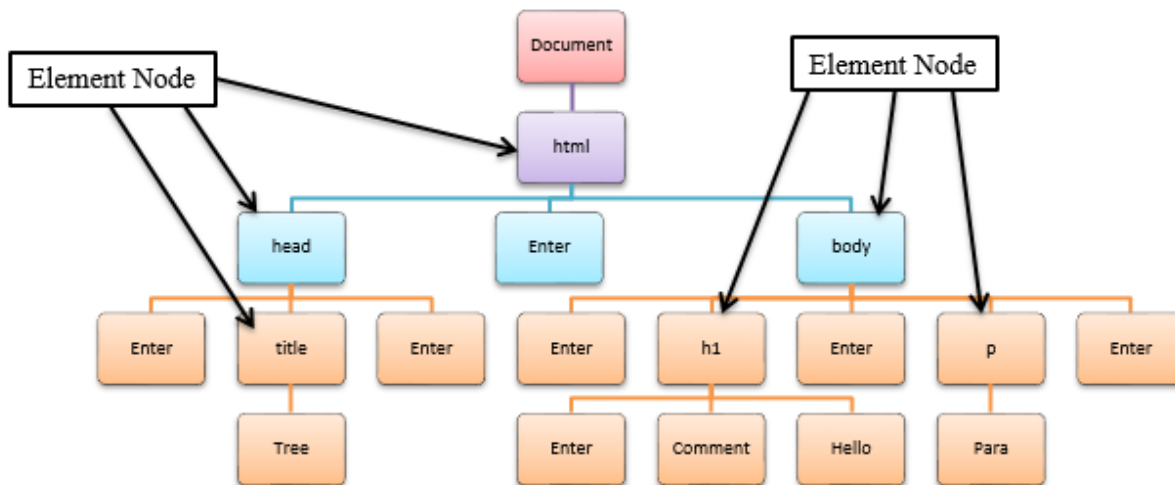
What is NodeList?

A NodeList is a simple list of DOM elements **or** a collection of nodes (DOM elements) you got from the webpage, but it's not a full, real array.

Types Of Nodes

Some important NodeType are:

- Element Node
- Text Node
- Comment Node
- Attribute Node
- Document Node
- Many more...



HTMLCollection vs NodeList

Feature	HTMLCollection	NodeList
Definition	A collection of HTML elements.	A collection of nodes (can include elements, text, comments).
Returned By	Methods like: • <code>document.getElementsByClassName()</code> • <code>document.getElementsByTagName()</code>	Methods like: • <code>document.querySelectorAll()</code> • <code>Node.childNodes</code>
Content Type	Contains only element nodes (<code>nodeType === 1</code>)	Can contain any node type , including text and comment nodes
Live or Static	Live – updates automatically when DOM changes	Static – does not update when DOM changes

Selecting HTML Elements

- document.getElementById("ID")
- document.getElementsByClassName("ClassName")
- document.getElementsByTagName("TagName")
- document.querySelector("id/class/tag")
- document.querySelectorAll("id/class/tag")

DOM Traversing (Walking)

Walking Through Nodes

- element.parentNode
- element.childNodes
- element.firstChild
- element.lastChild
- element.nextSibling
- element.previousSibling

Walking Through Elements

- element.parentElement
- element.children
- element.firstElementChild
- element.lastElementChild
- element.nextElementSibling
- element.previousElementSibling

Closest Ancestor Matching Selector

- element.closest(selector)

Manipulating HTML

Properties

Do You Know About The **\$0**
Magic in Google Chrome
Inspect Tool 🧐🔍

- `element.tagName`
- `element.nodeName`

Manipulating Content

- `element.innerText`
- `element.textContent`
- `element.innerHTML`

Manipulating Attributes

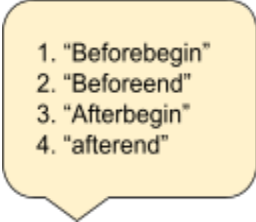
- `element.attributes`
- `element.getAttribute(attrName)`
- `element.setAttribute(attrName, value)`

Creating Elements

- `document.createElement("tagName")`

Inserting Elements

- `element.append(elem)`
- `element.prepend(elem)`
- `element.after(elem)`
- `element.before(elem)`
- `element.insertAdjacentElement("position", elem)`
- `element.insertBefore(newNode, referenceNode)`
- `parentElem.appendChild(elem)`



1. "Beforebegin"
2. "Beforeend"
3. "Afterbegin"
4. "afterend"

Replacing Elements

- `element.replaceWith(newElem)`
- `parentElem.replaceChild(newChild, oldChild)`

Deleting Elements

- `element.remove()`

- `parentElem.removeChild(elem)`

Manipulating CSS

Using Attribute Method

```
p.setAttribute("class", "change-bg");  
p.setAttribute("style", "padding: 10px; color: blue;");
```

Using Style Property

```
p.style.backgroundColor = "blue";  
p.style.color = 'white';
```

Using cssText Property

```
p.style.cssText = "font-size: 30px; color: red; background-color: black;";
```

Using className Property

```
p.className = "add-padding change-color change-bg";
```

Using classList Property

```
let classes = p.classList;  
  
// Methods  
p.classList.add("change-bg");  
p.classList.remove("change-bg");  
p.classList.toggle("add-padding");  
let bool = p.classList.contains("change-color");
```

Mastering Event Handling

You can handle an event using the following **three methods**:

- **Inline** Method (Inline JavaScript)
- **Property** Method
- **Listener** Method

1) Inline Method

```
<button onclick="console.log('button clicked ✓');">click me! 😊</button>
```

2) Property Method

```
let btn = document.querySelector("button");
btn.onclick = function () {
  alert("Hye! Muhammad Hussain 😊");
};
```

3) Listener Method

```
let btn = document.getElementsByTagName("button")[0];
btn.addEventListener('click', function() {
  console.log("Pakistan zindabad 🇵🇰");
});
```

How To Remove Event Listeners

```
let btn = document.querySelector("button");

const sayHello = () => {
  console.log("Hye! Muhammad Hussain 😊");
};

btn.addEventListener("click", sayHello); ← adding event listener
btn.removeEventListener("click", sayHello); ← removing event listener
```

If you want to add and then remove an event listener properly, you must:

- Use **addEventListener()**
- Use a named function (**function reference**), not an anonymous function.

Event Object

```
let btn = document.querySelector("button");
btn.addEventListener("click", (event) => {
  console.log(event);
});
```



event object

The event object is an **object** that is automatically passed to the **event handler function** when an event **occurs**.

Most Used Events

Mouse Events

Mouse Events	click	Fired when an element is clicked.
	dblclick	Triggered by a double click.
	mousedown	When mouse button is pressed.
	mouseup	When mouse button is released.
	mouseenter	Mouse enters an element (no bubbling).
	mouseleave	Mouse leaves an element (no bubbling).
	mouseover	Mouse moves over an element or its children.
	mouseout	Mouse leaves an element or its children.
	mousemove	Mouse is moved within an element.
	contextmenu	Right mouse button is clicked.

Keyboard Events

Keyboard Events	keydown	Key is pressed down.
	keypress	(Deprecated) Key that produces a character is pressed.
	keyup	Key is released.

Form Events

Form Events	submit	Form is submitted.
	reset	Form is reset.
	focus	Element receives focus.
	blur	Element loses focus.
	input	Value changes (real-time).
	change	Value of form element changes (on blur).

Touch Events

Touch Events	touchstart	Finger touches the screen.
	touchmove	Finger moves on screen.
	touchend	Finger is removed from screen.
	touchcancel	Touch interrupted (e.g., alert or system event).

Window Events

Window Events	load	Page and resources fully loaded.
	DOMContentLoaded	DOM is fully loaded (without waiting for styles/images).
	resize	Window is resized.
	scroll	Page is scrolled.

Clipboard Events

Clipboard Events	copy	Content is copied.
	cut	Content is cut.
	paste	Content is pasted.

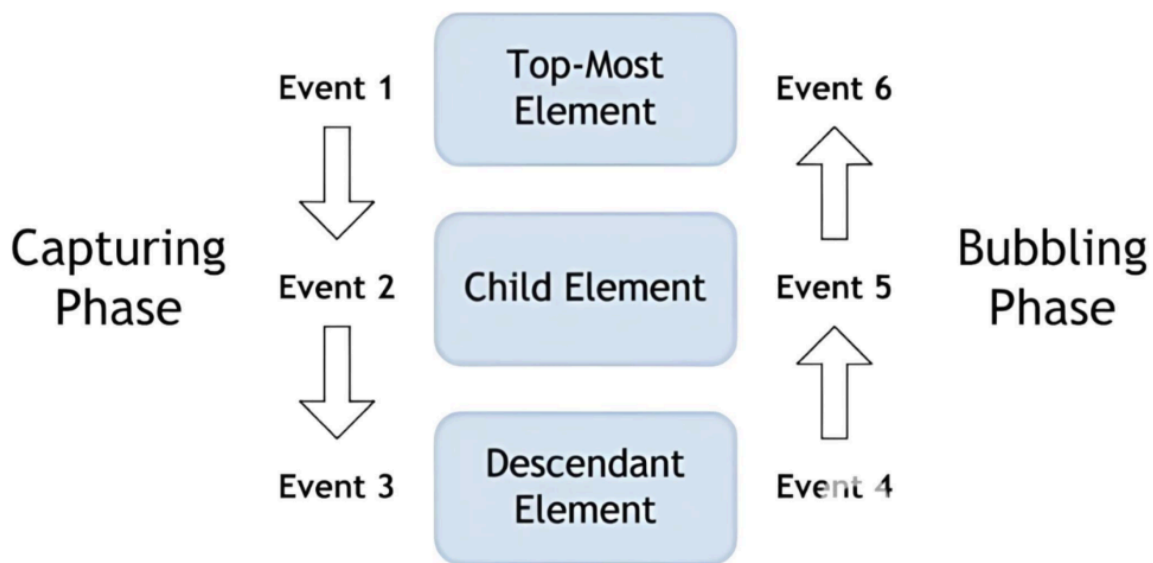
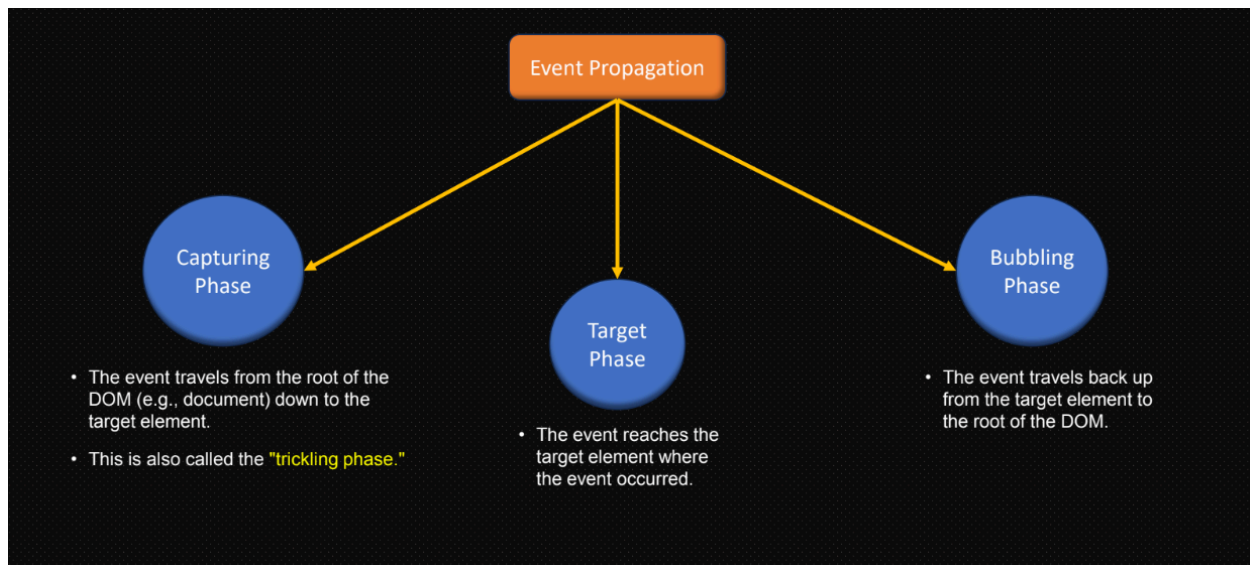
Mastering Event Lifecycle

Event Propagation

Event propagation describes the way events travel through the DOM after they are triggered. It consists of three phases:

- Capturing Phase

- Target Phase
- Bubbling Phase



👉 `event.target` v/s `event.currentTarget` v/s `this`

How to stop bubbling and capturing?

- `stopPropagation()`
- `stopImmediatePropagation()`

Event Delegation

🎯 Definition

Event Delegation is a technique where, instead of adding event listeners to individual child elements, you add a single event listener to a parent element and handle events for all children through event bubbling.

Mastering Numbers In JavaScript

There are **two ways** to declare a number variable

```
// Literal Method
let num = 12345;
let num1 = Number(12345);

// Constructor Method
let num2 = new Number(12345);
console.log(num2);
```

Number Properties

Property	Description
Number.MAX_VALUE	Largest possible number
Number.MIN_VALUE	Smallest possible number (positive)
Number.POSITIVE_INFINITY	Infinity
Number.NEGATIVE_INFINITY	-Infinity
Number.NaN	"Not-a-Number"
Number.EPSILON	Smallest difference between numbers
Number.isNaN()	Checks if value is NaN

Number Methods

Method	Description
parseInt()	Parses string to integer
parseFloat()	Parses string to float
isNaN()	Checks if value is NaN
Number.isFinite()	Checks if number is finite

Number Instance Methods

Method	Description
.toFixed(n)	Formats number to n decimal places
.toExponential(n)	Converts to exponential notation
.toPrecision(n)	Formats to n total digits
.toString()	Converts number to string
.valueOf()	Returns primitive value of Number object

“npm install [decimal.js](#)” Library

Mastering Math Object In JavaScript

JavaScript provides a **built-in Math Object** for mathematical operations.

Method	Description
Math.round(x)	Rounds to nearest integer
Math.floor(x)	Rounds down
Math.ceil(x)	Rounds up
Math.abs(x)	Absolute value
Math.sqrt(x)	Square root
Math.pow(x, y)	x to the power of y
Math.min(...args)	Smallest number
Math.max(...args)	Largest number
Math.random()	Random number between 0 (inclusive) and 1 (exclusive)

Generate Random Numbers

```
// Random number between 0 and 1
let randomNumber = Math.random();

// Random number between min and max (inclusive)
function generateRandomNumber(min, max) {
  let number = Math.floor(Math.random() * (max - min + 1)) + min;
  return number;
}

console.log(generateRandomNumber(5, 15));
```

Mastering Date Object In JavaScript

Date Fundamentals

- **GMT** (Greenwich Mean Time)
- **UTC** (Coordinated Universal Time)
- **ISO Standard** (International Organization for Standardization)
- **Unix Time** And **Unix Epoch**
- How do computers keep track of time?
- **Time Zones**.

Term	Meaning
Time Zone	Local time difference from UTC
GMT	Historical time standard
UTC	Global, accurate time reference
ISO Format	Standard format for date-time
Unix Time	Time counted from Jan 1, 1970
Epoch	Starting reference time for systems

Creating Date Object

```
// 1) No arguments (current date & time)
let date = new Date();

// 2) Date string (ISO or other formats)
let date1 = new Date("2025-09-09T18:15:00Z");
let date2 = new Date("Sep 9, 2025 8:14:00Z");
let date3 = new Date("2025/09/09 18:17:00Z");

// 3) Date numbers
let date4 = new Date(2025, 8, 9, 18, 22, 0);

// 4) Date milliseconds
let date5 = new Date(1747632000000);

// 5) Copy another date
let date6 = new Date("2005/05/27 12:00:00Z");
let copyDate = new Date(date6);
```

Date Get Methods

- `date.getFullYear()`
- `date.getMonth()`
- `date.getDate()`
- `date.getDay()`
- `date.getHours()`
- `date.getMinutes()`
- `date.getSeconds()`
- `date.getMilliseconds()`
- `date.getTime()`
- `date.getTimezoneOffset()` // UTC - Local Date

Date Set Methods

- `date.setFullYear(year, [month], [date])`
- `date.setMonth(month, [date])`
- `date.setDate(date)`
- `date.setHours(hour, [minutes], [seconds], [ms])`
- `date.setMinutes(minutes, [seconds], [ms])`
- `date.setSeconds(seconds, [ms])`
- `date.setMilliseconds(ms)`
- `date.setTime(time)`

Working With Timestamps

```
// 1st method
let date = Date.now();

// 2nd method
let date1 = new Date().getTime();
```

Formatting Dates Methods

- `date.toISOString();`
- `date.toString();`

- `date.toString();`
- `date.toTimeString();`
- `date.toLocaleString();`
- `date.toLocaleDateString();`
- `date.toLocaleTimeString();`



<https://www.unixtimestamp.com/>

Conversion From Timestamps to Date (Vice Versa)

```
// Timestamps to Date
let ts = Date.now();
let date = new Date(ts);
console.log(date);

// Date to Timestamps
let d = new Date();
let timestamp = d.getTime();
console.log(timestamp);
```

“npm install luxon” Library

Locale & Options

```
const date = new Date();

date.toLocaleString(locale, options) // Full date + time
date.toLocaleDateString(locale, options) // Date only
date.toLocaleTimeString(locale, options) // Time only
// locale --> en-IN, en-GB, fr-Fr, hi-En
```

Option	Possible Values
weekday	"long", "short", "narrow"
year	"numeric", "2-digit"
month	"numeric", "2-digit", "long", "short"
day	"numeric", "2-digit"
hour	"numeric", "2-digit"
minute	"numeric", "2-digit"
second	"numeric", "2-digit"
hour12	true or false
timeZone	"UTC", "Asia/Kolkata", etc.
timeZoneName	"short", "long"

Programming Paradigm In JavaScript

A **programming paradigm** 🎯 is a style or way of programming, essentially a set of principles, concepts, and practices that guide how software is designed and written. Each paradigm offers a distinct approach to problem-solving and code organization.

Programming Paradigm Includes

- **Imperative** programming (writing step-by-step instructions)
- **Declarative** Programming (Focuses on what to do, not how to do it.)
- **Event-driven** programming (responding to events like clicks or messages)
- **Functional** programming (using functions as the core building blocks)
- **Object-oriented** programming (organizing code around objects)

Paradigm	Common in JS?	Description
1. Imperative Programming	✅ Very common	Writing step-by-step logic like loops, conditionals, and variable updates.
2. Declarative Programming	✅ Very common	What it is: Focuses on what to do, not how to do it. Seen in Array.map, JSX in React, etc.
3. Event-Driven Programming	✅ Core in JS	Central to DOM events, and browser interactivity.
4. Functional Programming	✅ Increasingly popular	Emphasized in modern JS with arrow functions, map, filter, reduce, immutability, etc and avoid shared state & side effects etc.
5. Object-Oriented Programming	✅ Still widely used	JavaScript supports both prototype-based and class-based OOP. Used in frameworks and app architecture.

Imperative v/s Declarative

Imperative Programming	v/s	Declarative Programming:
Concept: You tell the computer how to do something step by step. Think of it like: Giving someone exact instructions to make a sandwich — one step at a time. <pre>let numbers = [1, 2, 3]; let doubled = []; for (let i = 0; i < numbers.length; i++) { doubled.push(numbers[i] * 2); } console.log(doubled); // [2, 4, 6]</pre>		Concept: You describe what you want, not how to do it. Think of it like: Ordering a coffee — you just say "I want a cappuccino", you don't explain how to make it. <pre>let numbers = [1, 2, 3]; let doubled = numbers.map(num => num * 2); console.log(doubled); // [2, 4, 6]</pre>

Further Categorization Of Some Important Paradigms

Programming Paradigm:
- Imperative Paradigm - Procedural Programming - Structured Programming - Declarative Paradigm - Functional Programming - Reactive Programming (e.g., RxJS) - Object-Oriented Programming - Class-based - Prototype-based - Event-Driven Programming - Based on event listeners, callbacks, DOM events, etc. - Asynchronous Programming (cross-paradigm) - Callback-based - Promise-based - async/await (syntactic sugar over Promises)

Procedural Programming

```
1 const consoleZero = () => console.log("number is zero.");
2 const consolePositive = () => console.log("number is positive.");
3 const consoleNegative = () => console.log("number is negative.");
4 const consoleProgramEnd = () => console.log("program is end.");
5
6 function checkNumber(num) {
7   if (num === 0) {
8     consoleZero();
9   } else if (num > 0) {
10    consolePositive();
11   } else {
12    consoleNegative();
13   }
14
15   consoleProgramEnd();
16 }
17
18 let num = 7;
19 checkNumber(num);
```

Structural Programming

```
1 function checkNumber(num) {
2   if (num === 0) {
3     console.log("number is zero.");
4   } else if (num > 0) {
5     console.log("number is positive.");
6   } else {
7     console.log("number is negative");
8   }
9
10  console.log("program is end.");
11 }
12
13 let num = -7;
14 checkNumber(num);
```

Event-Driven Programming

- **Concept** 🎯: Code responds to events like button clicks, form submissions, or messages.
- **Think of it like** 🚀: A doorbell — your action (pressing the bell) triggers a response (someone answers).

```
1 <button onclick="sayHello()">Click me!</button>
2
3 <script>
4   function sayHello() {
5     console.log("Hye! Muhammad Hussain");
6   }
7 </script>
```

Functional Programming

- **Concept** 🎯: Uses pure functions, avoids changing variables, and focuses on data transformation and immutability.
- **Think of it like** 🚀: A machine that always gives the same output for the same input.

```
1 function add(a, b) {
2   return a + b;
3 }
4
5 let res = add(10, 20); // 30
6 let res1 = add(60, 40); // 100
7
8 console.log(res);
9 console.log(res1);
```

Object-Oriented Programming

- **Concept** 🎯: Organize code into objects with properties (data) and methods (behavior).
- **Think of it like** 🚀: A car object — it has properties (color, brand) and methods (drive()).



```
1 class Car {
2   constructor(brand) {
3     this.brand = brand;
4   }
5
6   start() {
7     console.log(`${this.brand} is starting...`);
8   }
9 }
10
11 let myCar = new Car("Honda City");
12 console.log(myCar);
13 myCar.start();
```

Functional Programming In JavaScript

Functional Programming is a programming paradigm where functions are first-class citizens, and the focus is on pure functions, immutability, and function composition, rather than shared state and side effects.

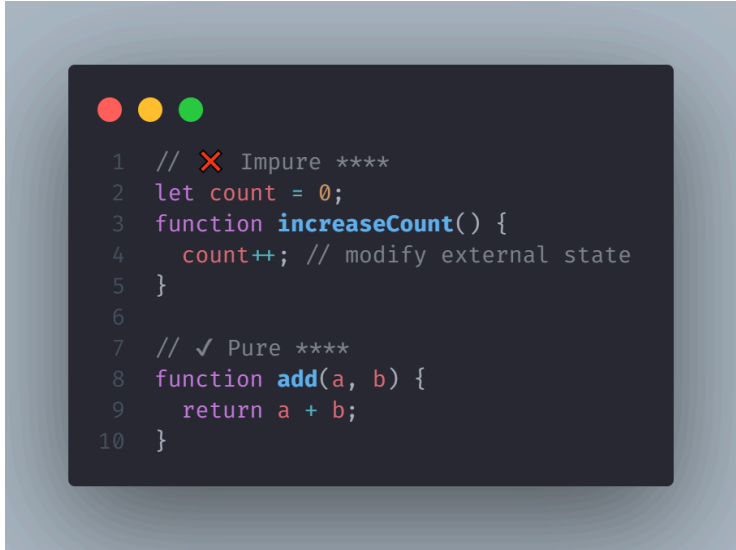
- Pure Functions
- Immutability
- Declarative
- Avoid shared state
- Avoid Side Effects
- Reuse or Compose Logic
- Don't Iterate
- Loose coupling

- First-Class & Higher-Order Functions

Pure Functions

A function is pure if:

- It returns the same output for the same input.
- It doesn't cause the side effects (like modifying external var or DOM)



```
1 // ✗ Impure ****
2 let count = 0;
3 function increaseCount() {
4   count++; // modify external state
5 }
6
7 // ✓ Pure ****
8 function add(a, b) {
9   return a + b;
10 }
```

Immutability

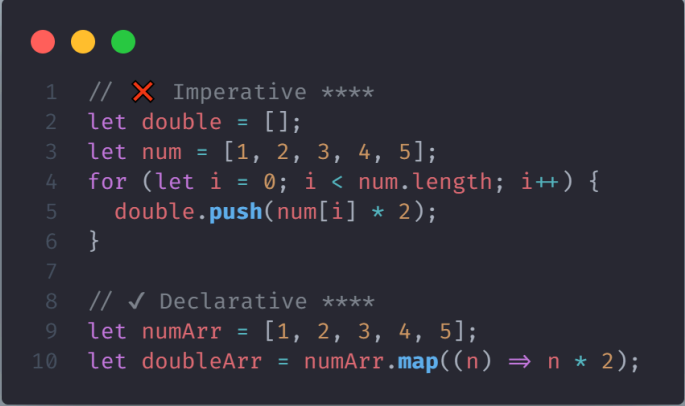
Do not modify the existing data. Instead, return a new copy.



```
1 // ✗ Mutable ****
2 let user = { name: "Hussain", age: 20 };
3 user.city = "Lahore";
4
5 // ✓ Immutable ****
6 let user = { name: "Hussain", age: 20 };
7 const updatedUser = { ...user, city: "Lahore" };
```

Declarative Code

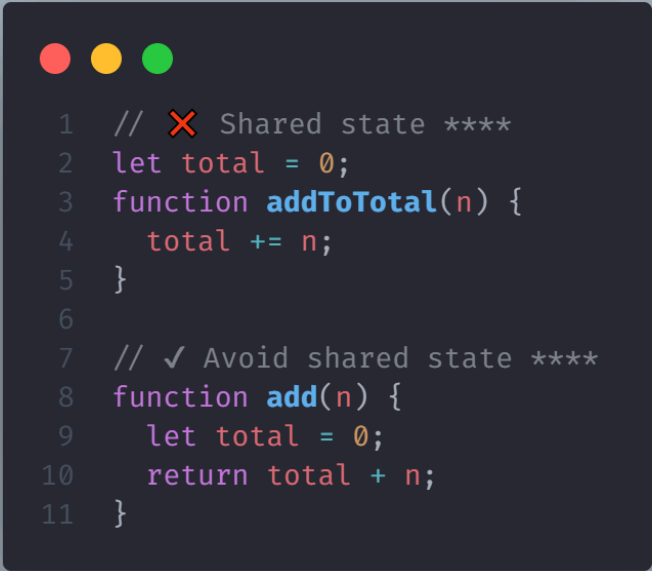
Describe what should be done, not how.



```
1 // ✗ Imperative ****
2 let double = [];
3 let num = [1, 2, 3, 4, 5];
4 for (let i = 0; i < num.length; i++) {
5   double.push(num[i] * 2);
6 }
7
8 // ✓ Declarative ****
9 let numArr = [1, 2, 3, 4, 5];
10 let doubleArr = numArr.map((n) => n * 2);
```

Avoid Shared State

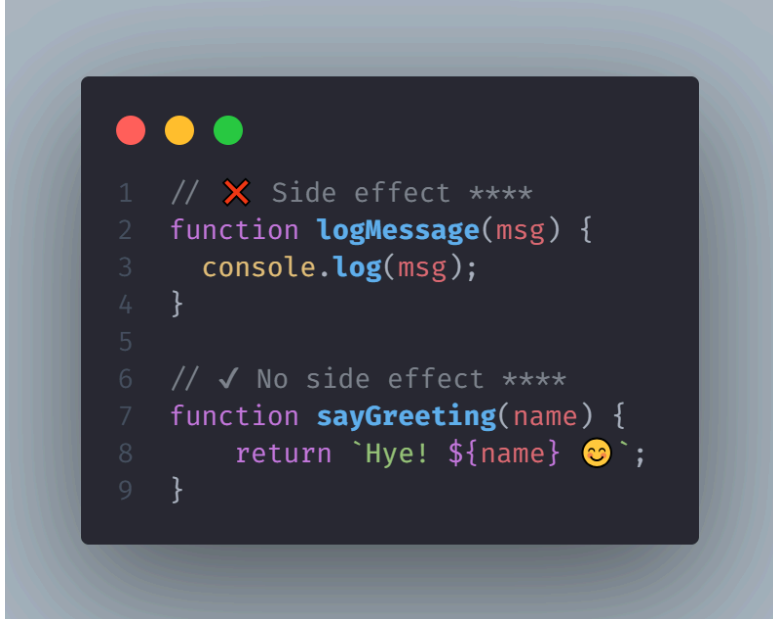
Shared mutable states can lead to bugs, especially in async or parallel system.



```
1 // ✗ Shared state ****
2 let total = 0;
3 function addToTotal(n) {
4   total += n;
5 }
6
7 // ✓ Avoid shared state ****
8 function add(n) {
9   let total = 0;
10  return total + n;
11 }
```

Avoid Side Effects

Side effects are anything a function does outside its scope.
(API call, DOM update, modifying global variables)



```
1 // ✗ Side effect ****
2 function logMessage(msg) {
3   console.log(msg);
4 }
5
6 // ✓ No side effect ****
7 function sayGreeting(name) {
8   return `Hye! ${name} 😊`;
9 }
```

Reuse Or Compose Logic

Build small reusable functions and compose them together.



```
1 const toLower = (str) => str.toLowerCase();
2 const removeSpaces = (str) => str.replaceAll(" ", "");
3 const atTheRate = (str) => `@${str}`;
4
5 let str = "Muhammad Hussain Ilyas";
6 let result = atTheRate(removeSpaces(toLower(str)));
7 console.log(result);
```

Don't Iterable (Imperatively)

Avoid for loops, while loops, etc. Use map, filter, and reduce methods.

```

1  let arr = [1, 2, 3, 4, 5, 6];
2
3  // ✗ Imperative style ****
4  let evens = [];
5  for (let n of arr) {
6    if (n % 2 === 0) {
7      evens.push(n);
8    }
9  }
10
11 // ✓ FP (Declarative) style
12 const evensArr = arr.filter((n) => n % 2 === 0);

```

Loose Coupling

Coupling refers to how dependent one piece of code is on another.

Loose coupling means less dependence. Keep functions and modules independent.

```

1  // ✗ Tightly coupled ****
2  function getUserData() {
3    return fetch("https://api.example.com/user").then((res) => res.json());
4  }
5
6  // ✓ Loosely coupled ****
7  function getData(api) {
8    return fetch(api).then((res) => res.json());
9  }

```

First Class v/s Higher Order, v/s Callback Functions

First Class Function

- Functions can be stored in variables, passed, and returned

Higher Order Function (HOF)

- Functions that take other functions as arguments or return them.

Callback Function

- Functions that are passed as arguments to the other functions (HOF).

Note:

- ❑ All callbacks are first-class functions, but not all first-class functions are callbacks.
- ❑ All higher order functions are first class functions but not all first-class functions are higher order functions

 $CF \subset HOF \subset FCF$

```
1 // 1. First-class function (FCF)
2 // We assign a function to a variable (treating it like a value)
3 const greet = function(name) {
4   return `Hello, ${name}!`;
5 };
6
7 // 2. Higher-order function (HOF)
8 // Takes another function as an argument (or returns one)
9 function processUserInput(callback) {
10   const name = "Alice";
11   // 3. Callback function (CF)
12   // The "greet" function is passed in and executed later
13   console.log(callback(name));
14 }
15
16 // Using all together:
17 processUserInput(greet);
```

Compose v/s Pipe

What is compose

- Right-to-left function composition.
- The last function in the argument list runs first.
- $\text{compose}(f, g)(x) \rightarrow$ is like $f(g(x))$

What is Pipe

- Left-to-right function composition.
- The first function in the argument list runs first.
- $\text{pipe}(f, g)(x) \rightarrow$ is like $g(f(x))$

```
1  const add2 = (num) => num + 2;
2  const multiple5 = (num) => num * 5;
3  const subtract3 = (num) => num - 3;
4
5  function compose( ... fns ) {
6    return function (val) {
7      return fns.reduceRight((acc, currFn) => {
8        return currFn(acc);
9      }, val);
10   };
11 }
12
13 function pipe( ... fns ) {
14   return function (val) {
15     return fns.reduce((acc, currFn) => {
16       return currFn(acc);
17     }, val);
18   };
19 }
20
21 // 1st method
22 let result = compose(subtract3, multiple5, add2)(5);
23 console.log(result);
24
25 // 2nd method
26 let result1 = pipe(subtract3, multiple5, add2);
27 console.log(result1(5));
```

Chaining

- Calling multiple methods in a single statement, one after another.

```

1 let str = "muha%^&$%^mmad@$$%hussai%$^*n";
2
3 function sanitizedStr(str) {
4   return str.replaceAll(/^[a-zA-Z0-9]/g, "").split("").reverse().join("");
5 }
6
7 let newStr = sanitizedStr(str);
8 console.log(newStr);

```

Master Object Oriented Programming

The different ways to create and use objects in JavaScript & these are the **foundations** for understanding **OOP** (Object Oriented Programming) in JS.

- Object literals
- Factory functions
- Constructor function
- Class syntax (ES6)

Object Literals

```

let student = {
  name: "Hussain",
  age: 20,
  greet: function () {
    return `Hye! My name is ${this.name}`;
  },
};

let result = student.greet();
console.log(result);

```

← properties

← Method

Factory Function

```
function createStudent(name, age) {
  return {
    name,
    age,
    greet: function () {
      return `Hye! I'm ${this.name}.`;
    },
  };
}

let student1 = createStudent("Hussain", 20);
let student2 = createStudent("Suleman", 21);

console.log(student1.greet());
console.log(student2.greet());
```

This technique doesn't include prototype
(unless you mentioned it manually)

Constructor Function

- Uses the **new** keyword
- This was the standard way to create an “**Object blueprints**” before ES6

```
function Student(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function () {
    return `Hye! I'm ${this.name}`;
  };
  return this;
}

const student1 = new Student("Hussain", 20);
const student2 = new Student("Suleman", 21);

console.log(student1);
console.log(student2);
```

Automatically set up a link
to prototype

Class Syntax (ES6)

- A modern, cleaner syntax for creating a constructor function

- Internally still works like a constructor function

```
class Student {
  constructor(name, age, city) {
    this.name = name;
    this.age = age;
    this.city = city;
    console.log("Constructor called");
  }
  displayInfo() {
    return `Name: ${this.name}, Age: ${this.age}, City: ${this.city}`;
  }
}

let s1 = new Student("Hussain", 20, "Lahore");
let s2 = new Student("Zaryab", 19, "Multan");

console.log(s1.displayInfo());
console.log(s2.displayInfo());
```

“this” Keyword

- Its value **depends** on how the **function** or **method** is called.
- In **OOP**, this refers to the **object** that is calling the **method**.
- It's **used** to **access** the current instance's **properties** or **methods**.
- **Arrow functions** don't have their **own** this — they inherit this from the surrounding (**lexical**) scope.

“new” Keyword

- In JavaScript, the new keyword is **used** to create an **instance** of an **object** from a **constructor function** or **class**.
- It's like saying: “Make me a new object from this blueprint (function or class).”

Prototypes In JS

- In JavaScript OOP, it allows us to **share methods** between all **instances** of a class or constructor function, making code memory-efficient.
- JavaScript is a **prototypal-based** (or prototype-based) language.

How does it work?

- Every object has an internal link to another object called its **prototype**.
- When you access a **property** or **method**, JavaScript looks for it in the object.
- If not found, it **climbs** the **prototype chain** to **find** it.

So, what about class in JS?

- JavaScript introduced the **class** keyword in **ES6**, but:
- class is just **syntactic sugar** — underneath, it's still using prototypes.

Object-Oriented Programming

- Object-Oriented Programming (**OOPs**) in JavaScript is a **programming paradigm** based on the **concept of objects**.
- These objects **encapsulate** both data (attributes, **properties**) and the functions that operate on that data (**methods**).
- JavaScript, while **not a purely class-based language** like Java or C++, is heavily object-oriented and supports **OOP** principles through its **prototype-based model** and **class syntax**.

What is Class?

- A Blueprint or Template, encapsulates data (properties) and functions (methods)

What is Object?

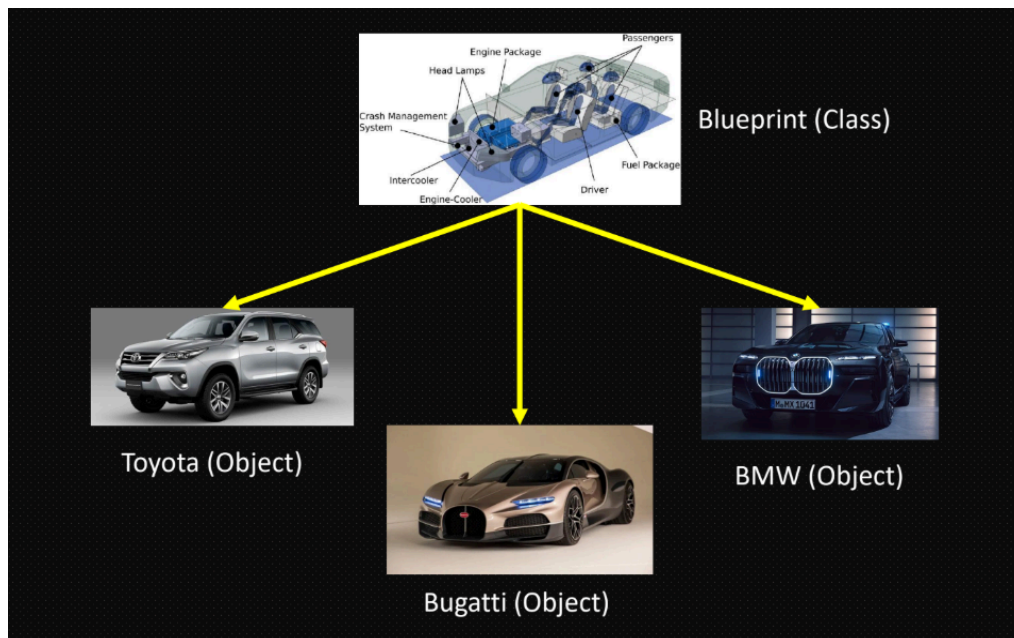
- instance of a class. Each object has its own unique set of values for its properties.

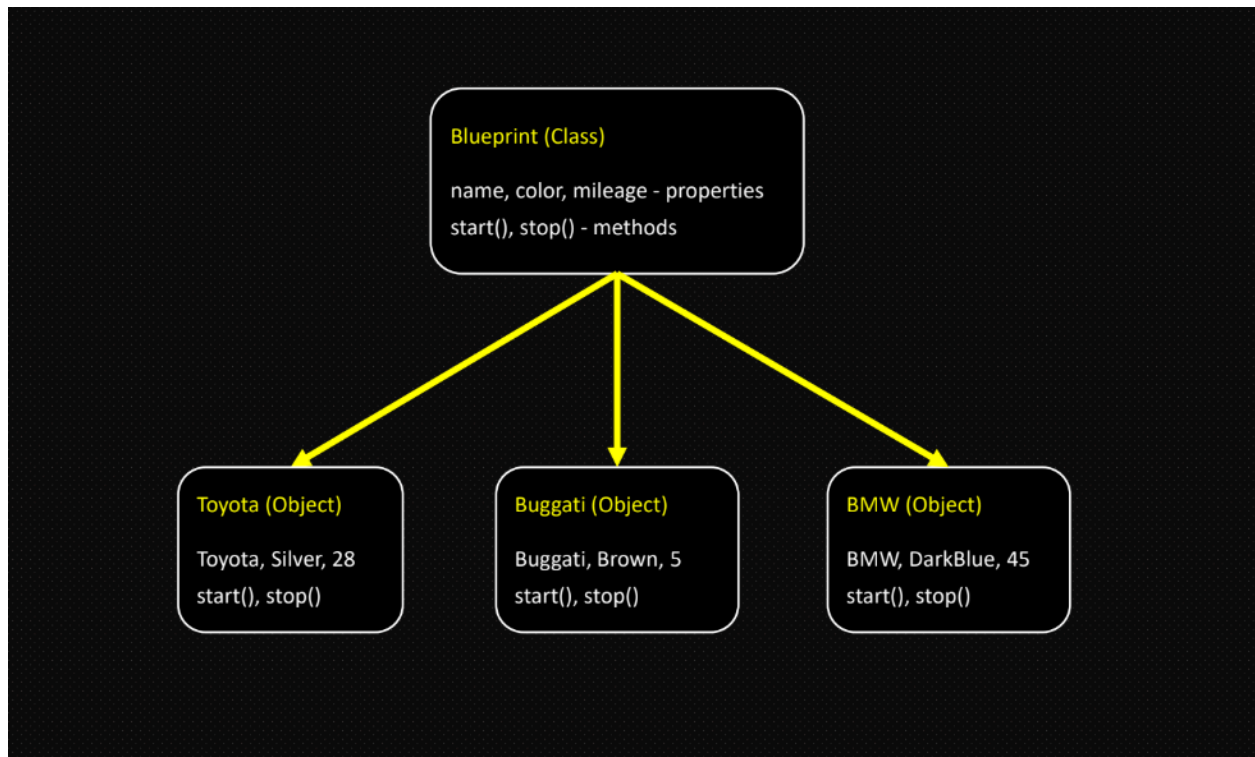
Constructor

- A constructor is a **special method** within a **class** that automatically called when a new object instance of that class is created.

- It is primarily used to **initialize object properties** with the **specific value** or perform setup task for the object.

```
1 class Car {
2   constructor(name, color, mileage) {
3     this.name = name;
4     this.color = color;
5     this.mileage = mileage;
6   }
7   carStart() {
8     return `${this.name} is starting...`;
9   }
10  carStop() {
11    return `${this.name} is stoping...`;
12  }
13 }
14
15 let bmw = new Car("BMW", "Black", 1000);
16 let kia = new Car("KIA", "White", 400);
17 let buggati = new Car("Buggati", "Navyblue", 5);
18
19 console.log(bmw.carStart());
20 console.log(kia.carStop());
```





4 Pillars Of OOP

- **Abstraction** - hiding complexity and showing only the essential features.
- **Encapsulation** - hiding data inside objects and providing security.
- **Inheritance** - using properties and methods from another object/class.
- **Polymorphism** - same method behaving differently based on the object.

1. Abstraction

- Showing only essential details, hiding complexity
- Abstraction means exposing only necessary methods and hiding unnecessary details

2. Encapsulation

- Hiding data inside a class
- Encapsulation means restricting direct access to some data and only allowing it through defined methods.

Code Example for Abstraction & Encapsulation

Example # 01

```
class Car {
  // Private field
  #fuel = 100;

  // hidden internal method
  #burnFuel() {
    this.#fuel -= 10;
  }

  start() {
    console.log("Car started");
    this.#burnFuel();
  }
}

let myCar = new Car();
myCar.start(); // Car started
console.log(myCar); // Car {#fuel: 90}
// myCar.#burnFuel(); // ❌ Not accessible
```

Example # 02

```
class Account {
  #balance = 0;
  constructor(balance) {
    this.#balance = balance;
  }
  #privateDetail() {
    console.log("My private details...");
  }
  getBalance() {
    this.#privateDetail();
    console.log("Balance: ", this.#balance);
  }
  setBalance(amount) {
    if (isNaN(amount)) {
      console.error("Invalid Amount!");
    } else {
      this.#balance = amount;
    }
  }
}

let myAccount = new Account(600);
// myAccount.#balance = 99; // ❌ Not Accessible
myAccount.setBalance("bxaxkbg"); // accessible but gives an error
myAccount.setBalance(700);
// myAccount.#privateDetail(); // ❌ Not Accessible
myAccount.getBalance();
```

Get & Set

```
class Account {
  #balance = 0;
  constructor(balance) {
    this.#balance = balance;
  }
  get balance() {
    return this.#balance;
  }
  set balance(balance) {
    if (isNaN(balance)) {
      console.error("Invalid Balance");
    } else {
      this.#balance = balance;
    }
  }
}

let myAccount = new Account(0);
myAccount.balance = "1000";
console.log(myAccount.balance);
```

Concept	What It Hides	What It Shows
Abstraction	The <i>process</i> / logic	A <i>simple interface</i>
Encapsulation	The <i>data</i> / internal state	Only what's allowed to access

3. Inheritance

- Inheritance is an OOP concept where one class(child) can acquire properties and methods of another class(parent).

Why Use Inheritance

- Reuse existing code.
- Create logical relationship
- Reduce duplication
- Easier maintenance and scalability

Example Code For Inheritance

```

class Car {
  constructor(name, color) {
    this.name = name;
    this.color = color;
  }
  start() {
    console.log(`${this.name} is starting ...`);
  }
  stop() {
    console.log(`${this.name} is stoping ...`);
  }
}

class ElectricCar extends Car {
  constructor(name, color, battery) {
    super(name, color);
    this.battery = battery;
  }
  charging() {
    console.log(`${this.name} is charging ...`);
  }
}

const myCar = new ElectricCar("BMW", "Navyblue", "Osaka");
myCar.start();
myCar.stop();
myCar.charging();

```

4. Polymorphism

- **Poly** = many, **morph** = forms, **polymorphism** = many forms.
- It allow different classes to define method with the same name but different behavior. (or we can say that has more than one form).

Imagine a **play button**

- On a **video**, it plays the video.
- On a **music player**, it plays the music
- On a **game**, it start the game

```

class MediaPlayer {
  play() { console.log("Play media ..."); }
}

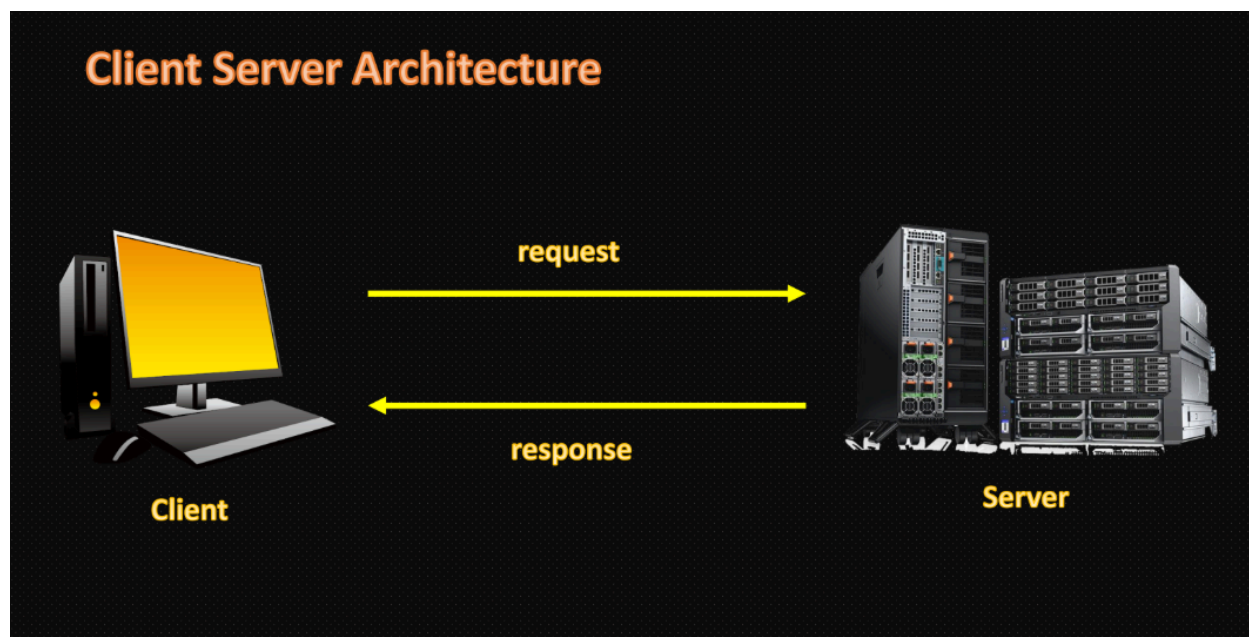
class Video extends MediaPlayer {
  play() { console.log("Play video ..."); }
}

class Audio extends MediaPlayer {
  play() { console.log("Play audio ..."); }
}

let vid = new Video();
let aud = new Audio();
vid.play();
aud.play();

```

Master APIs & JSON Response



XML v/s JSON

Features	JSON	XML
Data Format	Key-value pairs, arrays, objects	Nested tags with attributes & values
Readability	More concise, easy for humans	More verbose, harder to read quickly
Data Types	Supports numbers, strings, booleans, arrays, objects, and null	Everything is text (needs parsing for numbers/booleans)
Metadata	Limited (no attributes)	Supports attributes for extra info
Parsing	Fast, native in JavaScript	Slower, requires XML parsers
Schema Support	JSON Schema (simpler, less mature)	XML Schema (XSD, very powerful)
Comments	✗ Not supported	✓ Supported
Use Cases	APIs, web apps, config files	Legacy systems, documents, and industry standards (finance, healthcare, etc.)

JSON Example

```
{  
  "person": {  
    "name": "Alice",  
    "age": 30,  
    "isStudent": false  
  }  
}
```

XML Example

```
<person>  
  <name>Alice</name>  
  <age>30</age>  
  <isStudent>>false</isStudent>  
</person>
```

Understanding JSON(JavaScript Object Notation)

JSON

- ❑ JSON stands for JavaScript Object Notation.
- ❑ It's a lightweight data format used to store and exchange data — like sending data between a browser and a server.

```
// JavaScript Object  
const user = {  
  name: "Manas",  
  age: 25,  
  isStudent: true  
};  
  
// JSON version (as a string)  
const jsonUser = `{  
  "name": "Manas",  
  "age": 25,  
  "isStudent": true  
}`;
```

Note:

- ❑ Keys and string values are wrapped in double quotes.
- ❑ It's a string, not an object

Conversion Between JS and JSON

```
// Convert object to JSON
const jsonData = JSON.stringify(user);

// Convert JSON to object
const jsObject = JSON.parse(jsonData);
```

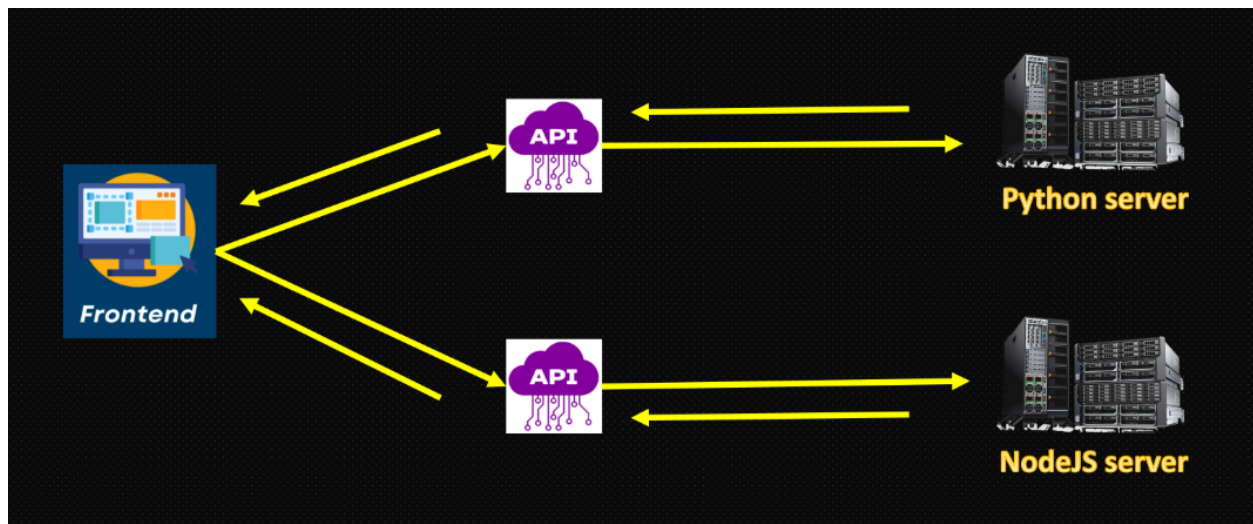
Use JSON.stringify() and JSON.parse():

- ❑ stringify() → Object to JSON string
- ❑ parse() → JSON string to Object

API (Application Programming Interface)

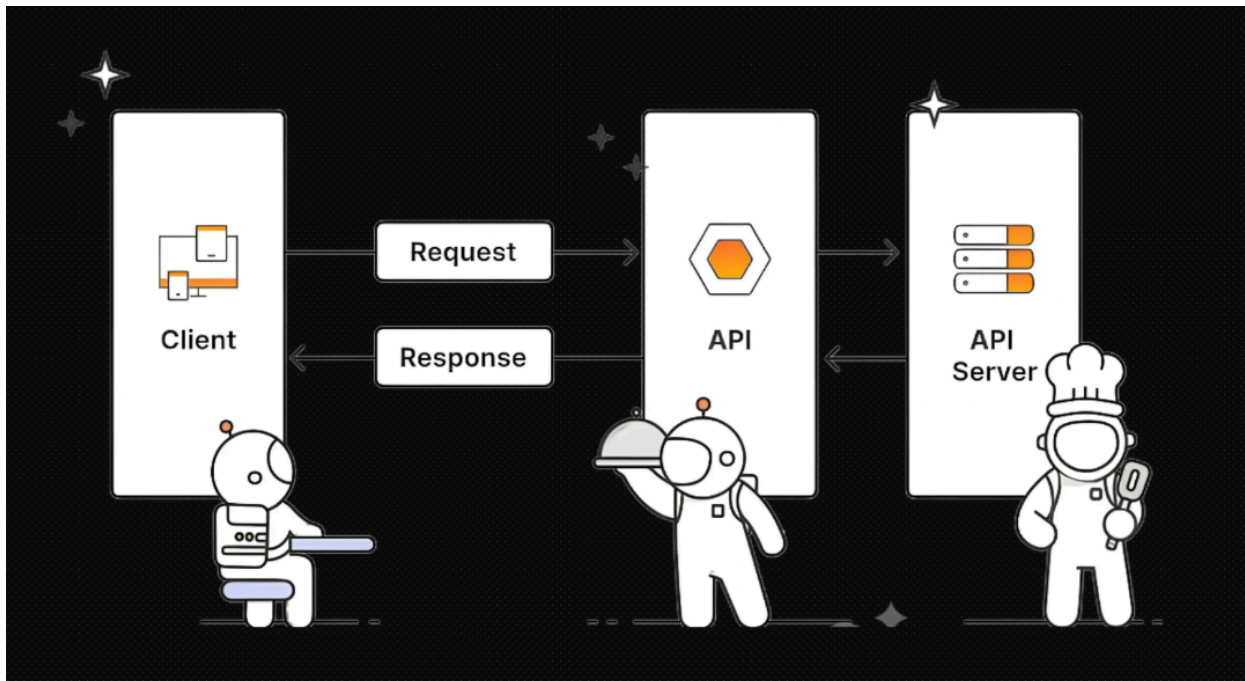
- API stands for Application Programming Interface.
- API is a set of **rules** and **protocols** that allows different software applications to communicate and **interact with each other**.

Communicate Between Different Languages/Systems



Uses Of APIs

- Communication between different languages/systems
- Controlled, secure access to data
- Monitoring & analytics
- Faster development with third-party services
- Automation & integration
- Microservices architecture
- Easy scaling and maintenance



Asynchronous Programming In JavaScript

There are two types of languages in nature

- Synchronous Programming
- Asynchronous Programming

Synchronous v/s Asynchronous

- **Synchronous** code runs **line by line**. Each operation must complete before the next one starts.
- **Asynchronous** code can start a task and move on **without waiting** for it to finish.

- **Asynchronous** code execution allows executing the next instructions (code) immediately and **doesn't block the flow**.

```
// Synchronous code
console.log("Task 01");
console.log("Task 02");
console.log("Task 03");
console.log("Task 04");

// Asynchronous Code
console.log("Task 01");
setTimeout(() => {
  console.log("Task 02");
}, 2000);
console.log("Task 03");
console.log("Task 04");
```

Feature	Synchronous	Asynchronous
Execution Flow	Line by line	Skips long tasks, comes back
Blocking	Yes	No
Use Cases	Simple tasks, calculations	API calls, DB queries, timers

Why Do We Get a Promise Instead Of Data

```
1 // It logs a promise, not actual data
2 let data = fetch("https://jsonplaceholder.typicode.com/users");
3 console.log(data);
```

We get a promise – not the real data – because the data is not ready yet.

API Calls Are Asynchronous

- Fetching data takes time (may be 500ms, 2s, or more)
- JavaScript does not want to stop anything and wait (It's single-threaded)
- So instead, it gives you a promise, saying

"I'll give you the data later, once it arrives."

Fetch

fetch() is a built-in JavaScript function used to make HTTP requests (like GET, POST, etc.) to a server or API. (It is like **"Hye server, please give me some data!"**).

What is CRUD

CRUD stands for

- Create
- Read
- Update
- Delete

These are the 4 basic operations we perform on data.

CRUD Operation	HTTP Method	Purpose
Create	POST	Add new data
Read	GET	Get/fetch existing data
Update	PUT / PATCH	Modify existing data
Delete	DELETE	Remove existing data

Callback

- A callback is a function that is passed as an argument to another function.
- Callback helps us to deal with tasks that take time, like loading data from a server, without blocking other code from running.

Example Of Callback Function

```

1  console.log("1. start fetching data ... ");
2
3  function fetchData(callback) {
4      setTimeout(() => {
5          console.log("2. Data fetched from server.");
6          callback();
7      }, 2000);
8  }
9
10 function processData() {
11     console.log("3. Now processing the data ... ");
12 }
13
14 fetchData(processData);
15
16 console.log("4. Do other thing while waiting ... ");

```

Callback Hell (Pyramid Of Doom)

- Callback Hell happens when you have many nested callback functions, one inside another, usually in asynchronous code.

```

function getData(data, callback) {
    setTimeout(() => {
        console.log(data);
        if (callback) {
            callback();
        }
    }, 2000);
}

console.log("fetching data ... ");

getData("1st data", () => {
    console.log("fetching data 02 ... ");
    getData("2nd data", () => {
        console.log("fetching data 03 ... ");
        getData("3rd data", () => {
            console.log("fetching data 04 ... ");
            getData("4th data", () => {
                console.log("fetching data 05");
                getData("5th data");
            });
        });
    });
});

```

← Callback Hell

How To Get API Data Before Promise Or Async/Await

```
1 // Real World Scenario of Callback
2 // Note:- Never try to understand 😊, if you want to live long
3
4 function getUsers(callback) {
5   let xhr = new XMLHttpRequest();
6   xhr.open("GET", "https://jsonplaceholder.typicode.com/users");
7
8   xhr.onload = function () {
9     if (xhr.status === 200) {
10       const user = JSON.parse(xhr.responseText);
11       callback(null, user);
12     } else {
13       callback(`Error: ${xhr.status}`);
14     }
15   };
16
17   xhr.onerror = function () {
18     callback("Network Error!");
19   };
20
21   xhr.send();
22 }
23
24 function showData(err, data) {
25   if (err) {
26     console.log("Failed:", err);
27   } else {
28     console.log("Success:", data);
29   }
30 }
31
32 getUsers(showData);
```

Promises

- A promise is a special **object** in JavaScript that represents a **task** that will finish in the **future**.

3 States Of Promise

A promise has three states

- **Pending** – still waiting
- **Resolved** (fulfilled) – task completed

- **Rejected** – something went wrong

```
function getData(data) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({
        name: data,
      });
    }, 2000);
  });
}

getData("Hussain")
  .then((res) => {
    console.log("Success:", res);
  })
  .catch((err) => {
    console.log("Reject", err);
  });
```

resolve and reject are callback functions provided by javascript

- **.then()** – Handle a **resolved** promise and pass the result forward
- **.catch()** – Handle any **rejected** promise (error)
- **.finally()** – Run code after completion (**success or error**)

Promise Chaining

```
function asyncTask(val) {
  return new Promise((res, rej) => {
    setTimeout(() => {
      res(val);
    }, 2000);
  });
}

asyncTask(10).then((res) => {
  console.log("step 01 result:", res);
  return res * 2;
}).then((res) => {
  console.log("step 02 result:", res);
  return res - 10;
}).then((res) => {
  console.log("step 03 result:", res);
  return res / 2;
}).then((res) => {
  console.log("step 04 result:", res);
}).catch((err) => {
  console.log("Rejected:", err);
}).finally(() => {
  console.log("Request Completed ... ");
})
```

← Promise Chaining

Async/Await

- Async/await helps you write **asynchronous code** in a **cleaner**, more **readable** way, almost as if it were **synchronous**.

async

- Makes a function always return a Promise

await

- Pauses inside an “async” function until the Promise is resolved

Example Code

```
// ===== 🚀 Example 01 =====  
  
function getData(name) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve(name);  
    }, 2000);  
  });  
}  
  
async function accessData(name) {  
  let response = await getData(name);  
  console.log(response);  
}  
  
accessData("Hussain Ilyas");  
  
// ===== 🚀 Example 02 =====  
  
async function fetchData(url) {  
  let response = await fetch(url);  
  let data = await response.json();  
  console.log(data);  
}  
  
fetchData("https://jsonplaceholder.typicode.com/users");
```

SetTimeout And SetInterval In JavaScript

1. setTimeout()

Executes a function **once** after a specified delay (in milliseconds).

It takes two parameters

- **callback:** The function to execute once after the delay.
- **delay:** Time in milliseconds (1000 ms = 1 second).

```
console.log("Start");

setTimeout(() => {
  console.log("Hello after 2 seconds");
}, 2000);

console.log("End");
```

2. setInterval()

Executes a function **repeatedly** at fixed time intervals.

It takes two parameters

- **callback:** The function to execute once after the delay.
- **delay:** Time in milliseconds (1000 ms = 1 second).

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log("Count:", count);

  if (count === 5) {
    clearInterval(intervalId); // stop after 5 times
  }
}, 1000);
```

Stopping the timers

- **clearTimeout(timeoutId)** — cancels a timer started with `setTimeout()`

- **clearInterval**(intervalId) – cancels a timer started with **setInterval**()

```
const timeoutId = setTimeout(() => console.log("Never runs"), 3000);
clearTimeout(timeoutId);
```

Local Storage And Session Storage

- Both **localStorage** and **sessionStorage** are part of the **web storage API** used to store key-value pairs in the browser.
- Store data in the browser (No server needed).
- Foundation for advanced topics (indexDB, Cookies, PWA).

Setting Data

```
// localStorage
localStorage.setItem("name", "Muhammad Hussain");

// sessionStorage
sessionStorage.setItem("name", "Muhammad Hussain");
```

Getting Data

```
// localStorage
let localData = localStorage.getItem("name");
console.log(localData);

// sessionStorage
let sessionData = sessionStorage.getItem("name");
console.log(sessionData);
```

Removing Data

```
// localStorage
localStorage.removeItem("name");

// sessionStorage
sessionStorage.removeItem("name");
```

Clearing All Data

```
// localStorage
localStorage.clear();

// sessionStorage
sessionStorage.clear();
```

You can also get or set the data using **dot notation** similar to the object

```
// localStorage
localStorage.name = "Muhammad Hussain";
console.log(localStorage.name);

// sessionStorage
sessionStorage.name = "Muhammad Hussain";
console.log(sessionStorage.name);
```

Storing Some Other Types Of Data (object, array, boolean, etc)

```
let user = {
  name: "Muhammad Hussain",
  city: "Lahore",
  age: 20,
};

localStorage.setItem("userData", JSON.stringify(user));

let userData = JSON.parse(localStorage.getItem("userData"));
console.log(userData);
```

Getting Key

```
let keyName = localStorage.key(0);
console.log(keyName);
```

Getting Length

```
let len = localStorage.length;  
console.log(len);
```

localStorage v/s sessionStorage

Feature	localStorage	sessionStorage
Scope	Shared across tabs and windows	Unique per tab/window
Lifetime	Persists even after browser close	Cleared when tab/window is closed
Storage Limit	~5–10 MB (per origin)	~5–10 MB (per origin)
Access	Same-origin only (not cross-site)	Same-origin only

Factors That Affect Storage

- Origin-based storage
- Browser limitation
- Data type limitation
- Security

Origin-Based Storage

- ❑ You cannot access localStorage or sessionStorage data across origins.

Data stored on <https://siteA.com> is not accessible on <https://siteB.com>.

- ❑ The data is only available to:

- That specific domain
- On that browser
- On that device
- In that user profile (Chrome profiles are separate)

- ❑ Even if you open “example.com” in Incognito Mode, its localStorage will be separate from the normal browser window.

Browser Limitations

- ❑ Most modern browsers allow around 5 to 10 MB per origin.
- ❑ Trying to exceed it throws a `QuotaExceededError`.

Data Type Limitations

- ❑ Only strings can be stored.
- ❑ You must `JSON.stringify()` complex data types (arrays, objects).

Security

- ❑ Not secure for sensitive data (passwords, tokens) — accessible via JavaScript.
- ❑ Vulnerable to Cross-Site Scripting (XSS) if not properly sanitized.

Spread Operator And Rest Operator

- Both use three **dots (...)** **syntax**, but do different things based on context
- **Spread operator** expands/unpacks values.
- **Rest operator** collects/packs values.

Spread Operator

1. Spread Arrays

```
let arr = [1, 2, 3, 4];
let newArr = [...arr, 5, 6];
console.log(newArr);
```

2. Copying Arrays (**Shallow Copy**)

```
let arr = [1, 2, 3, 4];
let copyArr = [...arr];
console.log(copyArr);
```

3. Merging Arrays

```
let boys = ["Hussain", "Suleman"];
let girls = ["Javeria", "Farah"];
let students = [...boys, ...girls];
console.log(students);
```

4. Spread String

```
let str = "Hussain";
let arr = [...str];
console.log(arr);
```

5. Spread in function calls

```
function add(a, b, c) {
  return a + b + c;
}

const nums = [1, 2, 3];
let result = add(...nums);
console.log(result);
```

6. Spread Objects

```
const user = { name: "Hussain", age: 20 };
const updateUser = { ...user, city: "Lahore" };
console.log(updateUser);
```

7. Merging Objects

```
const obj1 = { a: 1, b: 2 };
const obj2 = { x: 3, y: 4 };
const newObj = { ...obj1, ...obj2 };
console.log(newObj);
```

Rest Operator

1. Rest operator in function parameters

```
function sum( ... nums) {
  return nums.reduce((acc, num) => {
    return acc + num;
  }, 0);
}

console.log(sum(1, 2, 3, 5));
```

2. Rest operator in array destructuring

```
let [first, ...remaining] = [10, 20, 30, 40, 50];
console.log(first);
console.log(remaining);
```

3. Rest operator in object destructuring

```
let person = { user: "Suleman", age: 21, city: "Lahore" };
let {user, ...others} = person;
console.log(user);
console.log(others);
```

Rest Operator v/s Spread Operator

Feature	Spread ...	Rest ...
Purpose	Expands values	Collects values
Used in	Function calls, arrays, objects	Function parameters, destructuring
Position	Right side of = or in arguments	Left side of = or parameters
Example	add(...arr)	function add(...nums)

Destructuring in JavaScript

- Destructuring is a **JavaScript expression** used to **unpack** values from an **array**, or properties from an **object**, into distinct variables.

Array Destructuring

1. Basic destructuring

```
const numbers = [1, 2, 3, 4];  
let [num1, num2, num3, num4] = numbers;  
console.log(num1);  
console.log(num2);  
console.log(num3);  
console.log(num4);
```

2. Destructuring with rest operators

```
const numbers = [1, 2, 3, 4];  
let [num1, num2, ...others] = numbers;  
console.log(num1);  
console.log(num2);  
console.log(others);
```

3. Default Values

```
const numbers = [100];  
let [num1 = 1, num2 = 2] = numbers;  
console.log(num1); // 100  
console.log(num2); // 2
```

4. Skip array items

```
const numbers = [100, 200, 300];  
let [num1, , num3] = numbers;  
console.log(num1); // 100  
console.log(num3); // 300
```

5. Swapping Values

```
let a = 10, b = 20;
console.log(`a = ${a} & b = ${b}`);
[a, b] = [b, a];
console.log(`a = ${a} & b = ${b}`);
```

6. Nested Arrays

```
let users = ["Hussain", ["Suleman", "Zaryab"]];
let [user1, [user2, user3]] = users;
console.log(user1);
console.log(user2);
console.log(user3);
```

Object Destructuring

1. Basic destructuring

```
const obj = {fname: "Hussain", age: 20};
const {fname, age} = obj;
console.log(fname);
console.log(age);
```

2. Destructuring with rest operator

```
const obj = {fname: "Hussain", age: 20, city: "Lahore"};
const {fname, ...others} = obj;
console.log(fname);
console.log(others);
```

3. Default Values

```
const obj = {fname: "Hussain", age: 20, city: "Lahore"};
const {fname, grade = "A+"} = obj;
console.log(fname);
console.log(grade);
```

4. Renaming Variables

```
const obj = {fname: "Hussain", age: 20};  
const {fname: fullName, age: years} = obj;  
console.log(fullName);  
console.log(years);
```

5. Nested Destructuring

```
const user = {  
  id: 1,  
  profile: {  
    fname: "Hussain",  
    city: "Lahore",  
  },  
};  
  
let {id, profile: {fname, city}} = user;  
  
console.log(id);  
console.log(fname);  
console.log(city);
```

Function Destructuring

1. Array in parameters

```
function sum([a, b]) {  
  return a + b;  
}  
  
let arr = [100, 200];  
  
let result = sum(arr);  
console.log(result);
```

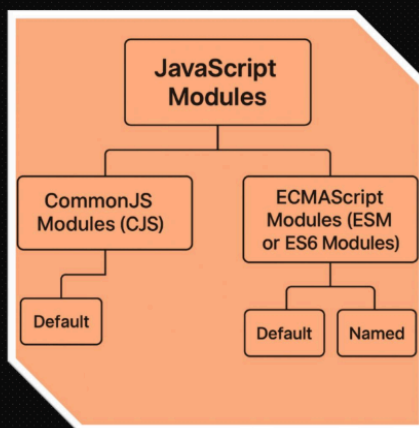
2. Object in parameters

```
function greet({name, age}) {
  return `I'm ${name}, mu age is ${age}`;
}

const user = {name: "Hussain", age: 20};

let result = greet(user);
console.log(result);
```

Modules In JavaScript



- ❑ Modules are a way to organize code into reusable files.
- ❑ They help you split your code into smaller, manageable parts, making it easier to maintain, reuse, and avoid name collisions.

Common JavaScript Modules

1. Export Single Item

Export single item

```
// utils.js
function greet(name) {
  console.log(`Hello, ${name}`);
}
module.exports = greet;
```



```
// script.js
const greetFunc = require('./utils');
greetFunc('Manas');
```

Note2:

- ❑ using file extension while importing is standard practice.
- ❑ Example: `const alpha = require('./utils.js')`

2. Export Multiple Items

Export multiple item

```
// script.js
const math = require('./math');
let result1 = math.add(2, 3); // 5
let result2 = math.subtract(10, 3); // 7
console.log(result1, result2)
```

```
// math.js
function add(a, b) {
  return a + b
}

const subtract = (a, b) => {
  return a - b;
}

module.exports = {
  add,
  subtract
};
```

ES6 Modules (Modern JavaScript)

1. Default Export

Default Export

```
// math.js
export default function add(a, b) {
  console.log(a * b);
}
```

```
// script.js
import defaultFunc from './math.js';

defaultFunc(5,2); // 10
```

Note2:
☐ Make Sure You're Using
 type: "module" in Node.js

2. Named Export

Named Export

```
// script.js
import {
  subtract,
  multiply,
  divide,
  modulo
} from './math.js';

subtract(5, 2);
multiply(5, 2);
divide(5, 2);
modulo(5, 2);
```

```
// math.js
export function subtract(a, b) {
  console.log(a - b);
}

export const multiply = (a, b) => {
  console.log(a * b);
}

function divide(a, b) {
  console.log(a / b);
}

function modulo(a, b) {
  console.log(a % b);
}

export {
  divide, modulo
}
```



How can we change the alias name while importing data?

```
import { sum as sumFunc, subtract, multiply } from './index.js';

console.log(sumFunc(10, 40));
console.log(subtract(40, 10));
console.log(multiply(10, 40));
```